

AWS EC2 Upgrade & Deployment Guide — Global ERP

Date: 2026-04-01 (updated) **Scope:** Upgrade existing EC2 instance + deploy recent changes (Mar 28 → Apr 1) + run 16 DB migrations **Assumption:** EC2 already running with Docker, Nginx, SSL, repo cloned, env files configured

Table of Contents

- 1. [Changes Summary \(Mar 28 → Apr 1\)](#)
- 2. [New Database Migrations \(16 files\)](#)
- 3. [Part A: Upgrade EC2 Instance](#)
- 4. [Part B: Deploy to DEV](#)
- 5. [Part C: Deploy to PROD](#)
- 6. [Post-Deployment Checks](#)
- 7. [Data Preservation — Live Data Safety](#)
- 8. [Rollback Plan](#)
- 9. [Quick Copy-Paste Commands](#)

1. Changes Summary

Commits (Mar 28 → Apr 1)

Commit	Description
<i>uncommitted</i>	Users page: department/branch/role filter dropdowns + employee <i>is_active</i> status
<i>uncommitted</i>	AI Inquiries: performance indexes + query rewrite
<i>uncommitted</i>	Sidebar: restructured navigation (categories: Main, Sales, Service Center, Jobs)
<i>uncommitted</i>	Call recording proxy + webhook " <i>recording</i> " key fix
<i>bab5f26</i>	Test — inquiries refactor, job card fixes, recording proxy, auth middleware
<i>26906b0</i>	Push changes — migration doc update
<i>042e59a</i>	Fix: add missing "recording" key to Yeastar webhook recording URL lookup
<i>93ea99a</i>	Revenue Command Center (RCC) — AI engine, lead sources, pipeline, leakage, marketing spend
<i>8a93ff2</i>	Add <i>tmp/*.log</i> to <i>.gitignore</i>
<i>7c320b8</i>	Test flows — PIS advisor portal, test panel, vendor bids, inspection print, invoice pay, GRN PDF
<i>67719ba</i>	Merge remote main
<i>09e6062</i>	Reflect real data — PIS config, advisor scores, lead distribution, commission, SLA

Commit	Description
457d208	Rename call center → sales center
961f747	Fix call center items
f515564	Fix recording issue
c59be47	Fix AIPanel auth + multi-provider AI config + e8 engine
265d23b	Call Center Performance Dashboard + e8 AI engine
e62a79c	Push
42f3732	Minor changes

Key Features

- **Revenue Command Center (RCC)** — 6 new pages + API routes + AI signal generator
- **PIS (Performance Incentive System)** — Advisor portal, lead distribution, commission, SLA
- **Multi-provider AI** — OpenAI + Anthropic dual provider
- **Call Center → Sales Center** rename
- **Sidebar restructure** — Simplified navigation categories (Main, Sales, Service Center, Jobs)
- **Users page filters** — Filter by department, branch, role + employee-level `is_active` status
- **AI Inquiries performance** — DB indexes + optimized queries for call recordings/phone matching
- **Call recording fix** — Recording URL lookup + proxy improvements
- **Test Panel** for flow testing
- **Vendor part details & bids** improvements
- **Service charges + main warehouse config**

2. New Database Migrations

16 new migrations (173 → 188):

#	File	What it does
173	173_multi_provider_ai_config.sql	Multi-provider AI config table
174	174_pis_config.sql	PIS system config
175	175_pis_advisor_scores.sql	Advisor scoring tables
176	176_pis_lead_distribution.sql	Lead distribution rules
177	177_pis_commission.sql	Commission structure
178	178_pis_sla_snapshots.sql	SLA snapshot tracking
179	179_pis_permissions.sql	PIS RBAC permissions
180	180_customers_insurance_name.sql	Customer insurance name field
181	181_advisor_portal_permission.sql	Advisor portal RBAC

#	File	What it does
182	182_vendor_part_details.sql	Vendor part details schema
183	183_main_warehouse_config.sql	Main warehouse config
184	184_service_charges_config.sql	Service charges config
185	185_rcc_marketing_spend.sql	RCC marketing spend table
186	186_rcc_permissions.sql	RCC permissions
187	187_employee_is_active.sql	Add is_active column to employees table
188	188_inquiries_performance.sql	Performance indexes for AI inquiries + customer phone matching

Migration runner auto-skips already-applied migrations via `schema_migrations` table.

Note on migration 187: Adds `is_active` boolean `NOT NULL DEFAULT true` to employees. All existing employees will default to active. No data loss.

Note on migration 188: Creates indexes only (no schema changes). May take a few seconds on large `customers` table due to expression indexes.

Part A: Upgrade EC2 Instance

A1. Check Current Instance & Decide Target Size

SSH into EC2:

```
ssh -i your-key.pem ubuntu@YOUR_EC2_IP
```

Check current resources:

```
# CPU & RAM
nproc
free -h

# Disk
df -h /

# OS version
lsb_release -a

# Docker version
docker --version
docker compose version

# Running containers
```

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
docker stats --no-stream
```

Recommended EC2 instance types:

Instance	vCPU	RAM	Use Case
t3.medium	2	4 GB	Minimum — tight for running dev+prod simultaneously
t3.large	2	8 GB	Recommended — comfortable for dev+prod+builds
t3.xlarge	4	16 GB	Ideal — fast builds, headroom for AI features

With 16 migrations (including performance indexes), RCC, PIS, sidebar restructure, and AI multi-provider — **t3.large (8GB)** is the sweet spot.

A2. Resize EC2 Instance (if needed)

Only needed if current instance is too small (e.g., t3.micro/small).

Via AWS Console:

- 1. Go to **EC2** → **Instances** → **Select your instance**
- 2. Click **Instance state** → **Stop instance** (wait for "Stopped")
- 3. Click **Actions** → **Instance settings** → **Change instance type**
- 4. Select **t3.large** (or your target)
- 5. Click **Apply**
- 6. Click **Instance state** → **Start instance**
- 7. Wait for status checks to pass

Via AWS CLI:

```
# From your local machine (not the EC2)
INSTANCE_ID="i-0xxxxxxxxxxxxx"

# Stop
aws ec2 stop-instances --instance-ids $INSTANCE_ID
aws ec2 wait instance-stopped --instance-ids $INSTANCE_ID

# Resize
aws ec2 modify-instance-attribute --instance-id $INSTANCE_ID --instance-type
'{"Value":"t3.large"}'

# Start
aws ec2 start-instances --instance-ids $INSTANCE_ID
aws ec2 wait instance-running --instance-ids $INSTANCE_ID

# Get new public IP (if using non-Elastic IP)
aws ec2 describe-instances --instance-ids $INSTANCE_ID \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text
```

If you use an Elastic IP, the IP stays the same after restart. If not, the public IP will change — update your DNS records.

A3. Expand EBS Volume (if disk is tight)

Check current disk:

```
df -h /  
# If less than 20GB free, expand
```

Expand via AWS Console:

1. Go to **EC2** → **Volumes** (or find volume from instance details)
2. Select the root volume → **Actions** → **Modify volume**
3. Increase size (e.g., 40GB → 80GB)
4. Click **Modify**

Then SSH in and grow the filesystem:

```
# Check partition name  
lsblk  
  
# Grow partition (usually /dev/xvda1 or /dev/nvme0n1p1)  
sudo growpart /dev/xvda 1  
# OR for nitro instances:  
sudo growpart /dev/nvme0n1 1  
  
# Resize filesystem  
sudo resize2fs /dev/xvda1  
# OR for XFS:  
sudo xfs_growfs /  
  
# Verify  
df -h /
```

A4. Update System & Docker on EC2

```
# SSH into EC2  
ssh -i your-key.pem ubuntu@YOUR_EC2_IP  
  
# Update OS packages  
sudo apt update && sudo apt upgrade -y  
sudo apt autoremove -y  
  
# Upgrade Docker if needed (should be 24+)  
docker --version  
# If outdated:
```

```
curl -fsSL https://get.docker.com | sh

# Verify Docker Compose V2
docker compose version
```

A5. Update Security Group (if needed)

Ensure these inbound rules exist in your EC2 Security Group:

Port	Protocol	Source	Purpose
22	TCP	Your IP	SSH
80	TCP	0.0.0.0/0	HTTP (Nginx redirect)
443	TCP	0.0.0.0/0	HTTPS (Nginx + SSL)

Ports 3000, 3001, 5432 should **NOT** be open to the internet — they're internal only (Nginx proxies to them).

A6. Clean Docker Space Before Build

```
# Check Docker disk usage
docker system df

# Remove old/dangling images (safe – only removes unused)
docker image prune -f

# More aggressive cleanup (removes all unused images)
docker image prune -a -f

# Check space after cleanup
df -h /
```

Part B: Deploy to DEV

B1. Pull Latest Code

```
cd /opt/global-erp
git fetch origin
git pull origin main
```

If you get ownership errors: `git config --global --add safe.directory /opt/global-erp`

B2. Backup DEV Database

```
mkdir -p /opt/backups

docker compose -p global-erp-dev -f docker-compose.dev.yml exec postgres \
  pg_dump -U autoguru global_erp_dev > /opt/backups/dev-$(date +%Y%m%d-%H%M%S).sql

# Verify backup
ls -lh /opt/backups/dev-*.sql | tail -1
```

B3. Rebuild & Start DEV

```
docker compose -p global-erp-dev -f docker-compose.dev.yml down
docker compose -p global-erp-dev -f docker-compose.dev.yml build --no-cache
docker compose -p global-erp-dev -f docker-compose.dev.yml up -d
```

B4. Run Migrations on DEV

```
docker compose -p global-erp-dev -f docker-compose.dev.yml exec web pnpm
db:migrate
```

Verify:

```
docker compose -p global-erp-dev -f docker-compose.dev.yml exec postgres \
  psql -U autoguru -d global_erp_dev -c "SELECT * FROM schema_migrations ORDER BY
id DESC LIMIT 15;"
```

Should show up to `186_rcc_permissions`.

B5. Verify DEV

```
docker compose -p global-erp-dev -f docker-compose.dev.yml ps
docker compose -p global-erp-dev -f docker-compose.dev.yml logs web --tail=30
curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1:3001
```

B6. DEV Smoke Tests

Open <https://dev.yourdomain.com>:

- ☐ Login works
- ☐ Sidebar restructured — categories: **Main, Sales, Service Center, Jobs**
- ☐ Sidebar shows **Revenue Command Center**
- ☐ Sidebar shows **Sales Center** (not "Call Center")
- ☐ PIS → Advisor Portal loads

- ☐ Settings → Users page: department/branch/role filter dropdowns work
 - ☐ Settings → Users page: status badge shows employee Active/Inactive (not user account)
 - ☐ Inspection → Estimate → Approval flow
 - ☐ Invoice creation + payment
 - ☐ Vendor → Part details + bids
 - ☐ AI Inquiries page loads fast (performance indexes applied)
 - ☐ Call recordings play correctly
 - ☐ AI Panel responds (multi-provider)
-

Part C: Deploy to PROD

Only after DEV smoke tests pass.

C1. Backup PROD Database (CRITICAL)

```
docker compose -p global-erp-prod -f docker-compose.prod.yml exec postgres \
  pg_dump -U autoguru global_erp_prod > /opt/backups/prod-$(date +%Y%m%d-
%H%M%S).sql

# Verify backup exists and has real data
ls -lh /opt/backups/prod-*.sql | tail -1
```

C2. Rebuild & Start PROD

```
docker compose -p global-erp-prod -f docker-compose.prod.yml down
docker compose -p global-erp-prod -f docker-compose.prod.yml build --no-cache
docker compose -p global-erp-prod -f docker-compose.prod.yml up -d
```

Downtime: ~3-8 minutes (Next.js prod build). Plan accordingly.

C3. Run Migrations on PROD

```
docker compose -p global-erp-prod -f docker-compose.prod.yml exec web pnpm
db:migrate
```

Verify:

```
docker compose -p global-erp-prod -f docker-compose.prod.yml exec postgres \
  psql -U autoguru -d global_erp_prod -c "SELECT * FROM schema_migrations ORDER BY
id DESC LIMIT 15;"
```

C4. Verify PROD


```
docker compose -p global-erp-prod -f docker-compose.prod.yml ps
docker compose -p global-erp-prod -f docker-compose.prod.yml logs web --tail=30
curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1:3000
curl -s -o /dev/null -w "%{http_code}" https://yourdomain.com
```

C5. PROD Smoke Tests

Open <https://yourdomain.com>:

- ☐ Login with real credentials
- ☐ Sidebar restructured correctly (Main, Sales, Service Center, Jobs)
- ☐ Revenue Command Center accessible + data loads
- ☐ Sales Center + WebSocket connection
- ☐ PIS Advisor Portal functional
- ☐ Settings → Users: filters + employee-level status
- ☐ Full flow: Inspection → Estimate → Approval → Invoice → Payment
- ☐ Vendor bids + part details
- ☐ AI Inquiries page loads fast
- ☐ Call recordings play correctly
- ☐ AI features respond
- ☐ No console errors

6. Post-Deployment Checks

```
# All containers running
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

# Resource usage (ensure no OOM)
docker stats --no-stream

# Disk space
df -h /

# Nginx healthy
sudo nginx -t && sudo systemctl status nginx

# Monitor prod logs for 5-10 mins
docker compose -p global-erp-prod -f docker-compose.prod.yml logs -f web
```

7. Data Preservation — Live Data Safety

Important: All live data (customers, cars, employees, users, companies) is stored in PostgreSQL volumes (`postgres_data` for prod, `postgres_dev_data` for dev). These volumes persist across container rebuilds.

What is SAFE (data is preserved):

Action	Data safe?	Why
<code>docker compose down</code>	Yes	Only stops containers, volumes remain
<code>docker compose build --no-cache</code>	Yes	Rebuilds app image only, not DB
<code>docker compose up -d</code>	Yes	Reconnects to existing volume
<code>pnpm db:migrate</code>	Yes	Only adds new tables/columns, never drops existing
<code>git pull</code>	Yes	Only updates code, not DB data

What is DANGEROUS (data loss risk):

Action	Risk
<code>docker compose down -v</code>	DELETES ALL VOLUMES including DB data
<code>docker volume rm postgres_data</code>	DELETES PROD DB
<code>docker system prune --volumes</code>	DELETES ALL UNUSED VOLUMES
Dropping/recreating DB manually	DELETES ALL DATA

Verify data is intact after deploy:

```
# PROD – check row counts for critical tables
docker compose -p global-erp-prod -f docker-compose.prod.yml exec postgres \
  psql -U autoguru -d global_erp_prod -c "
    SELECT 'companies' AS tbl, COUNT(*) FROM companies
    UNION ALL SELECT 'customers', COUNT(*) FROM customers
    UNION ALL SELECT 'cars', COUNT(*) FROM cars
    UNION ALL SELECT 'employees', COUNT(*) FROM employees
    UNION ALL SELECT 'users', COUNT(*) FROM users
    ORDER BY tbl;
"

# DEV – same check
docker compose -p global-erp-dev -f docker-compose.dev.yml exec postgres \
  psql -U autoguru -d global_erp_dev -c "
    SELECT 'companies' AS tbl, COUNT(*) FROM companies
    UNION ALL SELECT 'customers', COUNT(*) FROM customers
    UNION ALL SELECT 'cars', COUNT(*) FROM cars
    UNION ALL SELECT 'employees', COUNT(*) FROM employees
    UNION ALL SELECT 'users', COUNT(*) FROM users
    ORDER BY tbl;
"
```

Run these counts BEFORE and AFTER deploy to confirm no data was lost.

Backup critical tables individually (extra safety):

```
# Export just the critical tables (PROD)
for TABLE in companies customers cars employees users; do
  docker compose -p global-erp-prod -f docker-compose.prod.yml exec postgres \
    pg_dump -U autoguru -d global_erp_prod -t $TABLE --data-only \
    > /opt/backups/prod-{$TABLE}--$(date +%Y%m%d-%H%M%S).sql
done

# Verify exports
ls -lh /opt/backups/prod-{companies,customers,cars,employees,users}-*.sql | tail
-5
```

8. Rollback Plan

PROD Rollback Steps:

```
# 1. Stop prod
docker compose -p global-erp-prod -f docker-compose.prod.yml down

# 2. Go back to last known good commit
cd /opt/global-erp
git log --oneline -5
git checkout <good-commit>

# 3. Rebuild + start
docker compose -p global-erp-prod -f docker-compose.prod.yml build --no-cache
docker compose -p global-erp-prod -f docker-compose.prod.yml up -d

# 4. Restore database if migrations broke something
docker compose -p global-erp-prod -f docker-compose.prod.yml exec -T postgres \
  psql -U autoguru -d global_erp_prod < /opt/backups/prod-backup-YYYYMMDD-
HHMMSS.sql
```

Migrations are forward-only — restore from backup if needed.

9. Quick Copy-Paste Commands

Full Deploy Sequence (SSH into EC2, then run):

```
# ===== FULL DEPLOY — COPY & RUN STEP BY STEP =====

cd /opt/global-erp
```

```

git pull origin main
mkdir -p /opt/backups

# — DEV —————
docker compose -p global-erp-dev -f docker-compose.dev.yml exec postgres \
  pg_dump -U autoguru global_erp_dev > /opt/backups/dev-$(date +%Y%m%d-%H%M%S).sql

docker compose -p global-erp-dev -f docker-compose.dev.yml down
docker compose -p global-erp-dev -f docker-compose.dev.yml build --no-cache
docker compose -p global-erp-dev -f docker-compose.dev.yml up -d
docker compose -p global-erp-dev -f docker-compose.dev.yml exec web npm
db:migrate
docker compose -p global-erp-dev -f docker-compose.dev.yml ps

echo ">>> DEV deployed. Run smoke tests on dev.yourdomain.com <<<"

# — PROD (run after DEV smoke tests pass) —————
docker compose -p global-erp-prod -f docker-compose.prod.yml exec postgres \
  pg_dump -U autoguru global_erp_prod > /opt/backups/prod-$(date +%Y%m%d-
%H%M%S).sql

docker compose -p global-erp-prod -f docker-compose.prod.yml down
docker compose -p global-erp-prod -f docker-compose.prod.yml build --no-cache
docker compose -p global-erp-prod -f docker-compose.prod.yml up -d
docker compose -p global-erp-prod -f docker-compose.prod.yml exec web npm
db:migrate
docker compose -p global-erp-prod -f docker-compose.prod.yml ps

echo ">>> PROD deployed. Run smoke tests on yourdomain.com <<<"

# — VERIFY DATA (run after both deploys) —————
echo "=== PROD data counts ==="
docker compose -p global-erp-prod -f docker-compose.prod.yml exec postgres \
  psql -U autoguru -d global_erp_prod -c "
    SELECT 'companies' AS tbl, COUNT(*) FROM companies
    UNION ALL SELECT 'customers', COUNT(*) FROM customers
    UNION ALL SELECT 'cars', COUNT(*) FROM cars
    UNION ALL SELECT 'employees', COUNT(*) FROM employees
    UNION ALL SELECT 'users', COUNT(*) FROM users
    ORDER BY tbl;"

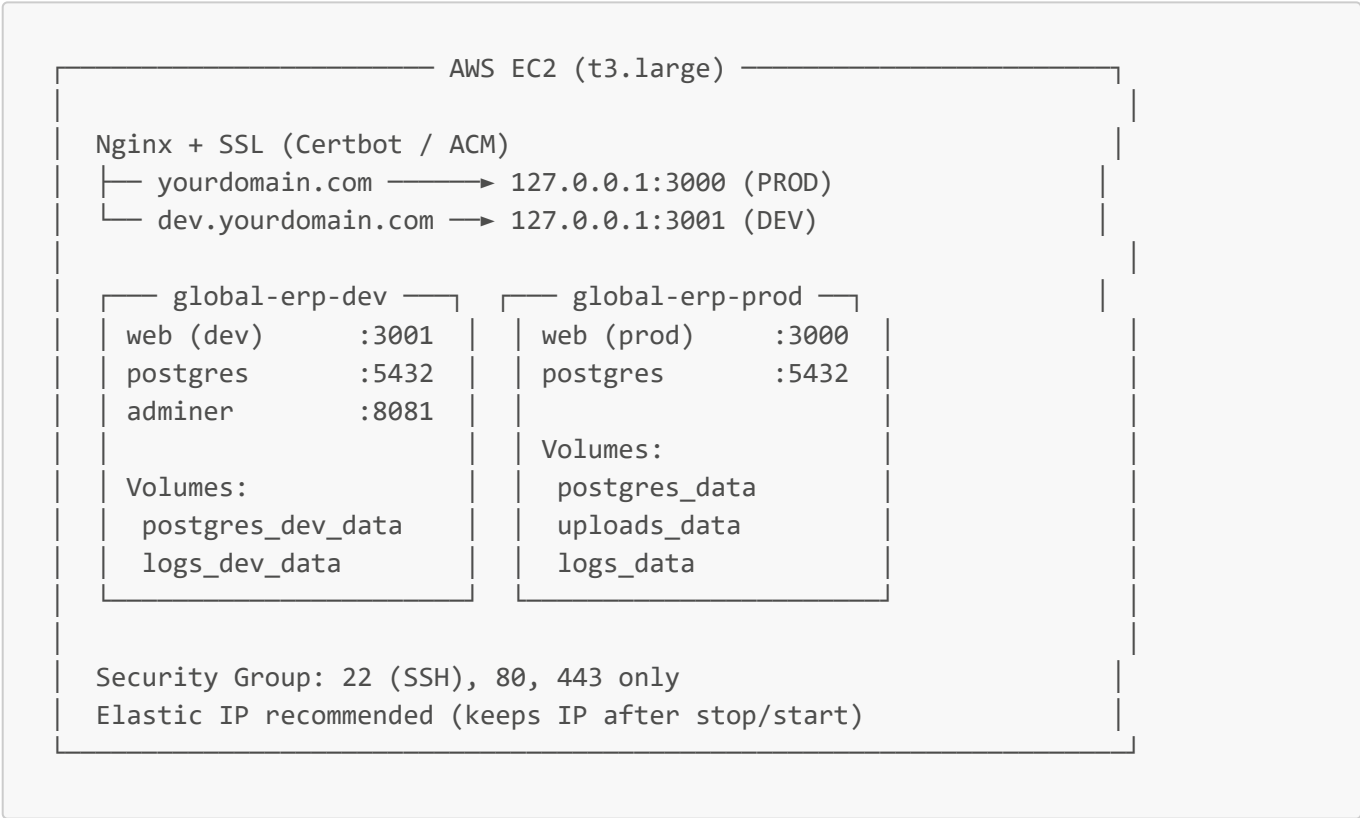
echo "=== DEV data counts ==="
docker compose -p global-erp-dev -f docker-compose.dev.yml exec postgres \
  psql -U autoguru -d global_erp_dev -c "
    SELECT 'companies' AS tbl, COUNT(*) FROM companies
    UNION ALL SELECT 'customers', COUNT(*) FROM customers
    UNION ALL SELECT 'cars', COUNT(*) FROM cars
    UNION ALL SELECT 'employees', COUNT(*) FROM employees
    UNION ALL SELECT 'users', COUNT(*) FROM users
    ORDER BY tbl;"

```

Useful Aliases (add to `~/ .bashrc`):

```
alias dev-up="cd /opt/global-erp && docker compose -p global-erp-dev -f docker-
compose.dev.yml up -d"
alias dev-down="cd /opt/global-erp && docker compose -p global-erp-dev -f docker-
compose.dev.yml down"
alias dev-logs="cd /opt/global-erp && docker compose -p global-erp-dev -f docker-
compose.dev.yml logs -f web"
alias dev-migrate="cd /opt/global-erp && docker compose -p global-erp-dev -f
docker-compose.dev.yml exec web pnpm db:migrate"
alias prod-up="cd /opt/global-erp && docker compose -p global-erp-prod -f docker-
compose.prod.yml up -d"
alias prod-down="cd /opt/global-erp && docker compose -p global-erp-prod -f
docker-compose.prod.yml down"
alias prod-logs="cd /opt/global-erp && docker compose -p global-erp-prod -f
docker-compose.prod.yml logs -f web"
alias prod-migrate="cd /opt/global-erp && docker compose -p global-erp-prod -f
docker-compose.prod.yml exec web pnpm db:migrate"
```

Architecture



Prepared: 2026-04-01 | **Applies to:** Commits [42f3732](#) → [bab5f26](#) + uncommitted (16 commits + uncommitted changes, 16 migrations)